

LAB 3

Classical Image Analysis:

Feature Matching, Edge Detection & Line Detection

Dữ liệu:

- lab3_stitch_left.jpg, lab3_stitch_right.jpg – cặp ảnh chụp cùng cảnh, có vùng chồng lấp nhau.
- lab3_scene.jpg – ảnh grayscale dùng cho phần 2 và 3.

Nội bài: File .ipynb có đầy đủ code chạy được, kết quả hình ảnh và nhận xét cho từng phần.

Phần 1. Image Stitching using Feature Point Matching

Cho hai ảnh chụp cùng một cảnh với vùng chồng lấp nhau. Nhiệm vụ là ghép hai ảnh thành một ảnh panorama hoàn chỉnh.

1.1. So sánh các bộ phát hiện đặc trưng

Thực hiện phát hiện keypoint và mô tả đặc trưng bằng **ít nhất 3** phương pháp sau: **ORB, SIFT, AKAZE, BRISK**.

Với mỗi phương pháp, báo cáo các chỉ số sau vào một bảng tổng hợp:

Phương pháp	Số kps	Số match (sau ratio test)	Inlier ratio (%)	Thời gian (ms)
ORB				
SIFT				
AKAZE				
BRISK				

Inlier ratio = tỉ lệ inlier sau RANSAC so với tổng số match trước RANSAC.

1.2. Ghép ảnh (Image Stitching)

Sử dụng phương pháp cho kết quả tốt nhất ở mục 1.1 để:

- Tìm các cặp điểm tương đồng bằng **BFMatcher** kết hợp **Lowe's ratio test** ($r = 0.75$).
- Tính **Homography matrix** H bằng RANSAC từ các cặp inlier.
- Ghép hai ảnh thành panorama bằng **warpPerspective**.

1.3. Yêu cầu trình bày

1. Hiển thị keypoint trên mỗi ảnh (với mỗi phương pháp).
 2. Vẽ các cặp match sau ratio test (dùng `drawMatches`).
 3. Hiển thị ảnh panorama kết quả cuối cùng.
 4. **Nhận xét:** phương pháp nào cho kết quả ghép ảnh tốt nhất và tại sao? Phân tích sự đánh đổi giữa tốc độ và chất lượng match.
-

Phần 2. Edge Detection

Sử dụng ảnh `lab3_scene.jpg`.

2.1. Phát hiện biên bằng Sobel và Canny

1. Làm mịn ảnh bằng **Gaussian Blur** với kernel size 3×3 và 7×7 .
2. Tính gradient bằng **Sobel**:
 - Tính G_x , G_y riêng biệt, hiển thị từng hướng.
 - Tổng hợp: $G = \sqrt{G_x^2 + G_y^2}$, áp dụng ngưỡng để thu biên.
3. Phát hiện biên bằng **Canny** với ít nhất 2 bộ ngưỡng khác nhau.

2.2. So sánh bản chất Sobel và Canny

1. Hiển thị cạnh nhau: ảnh gốc – Sobel – Canny (cùng mức Gaussian trước đó).
 2. **Nhận xét** về sự khác biệt bản chất:
 - Sobel chỉ tính gradient – biên thu được dày hay mỏng? Tại sao?
 - Canny bổ sung *non-maximum suppression* và *hysteresis thresholding* – điều này thể hiện như thế nào trên kết quả?
 - Trường hợp nào nên dùng Sobel, trường hợp nào nên dùng Canny?
 3. Phân tích ảnh hưởng của Gaussian Blur trước khi lọc biên đối với mỗi phương pháp.
-

Phần 3. Line Detection

Sử dụng ảnh `lab3_scene.jpg`. Dùng kết quả Canny từ Phần 2 làm đầu vào.

3.1. Phát hiện đường thẳng bằng tích chập (Convolution Kernel)

Áp dụng các **line detection kernel** để phát hiện đường thẳng theo hướng:

$$K_{\text{horizontal}} = \begin{bmatrix} -1 & -1 & -1 \\ 2 & 2 & 2 \\ -1 & -1 & -1 \end{bmatrix}, \quad K_{\text{vertical}} = \begin{bmatrix} -1 & 2 & -1 \\ -1 & 2 & -1 \\ -1 & 2 & -1 \end{bmatrix}$$

$$K_{45} = \begin{bmatrix} -1 & -1 & 2 \\ -1 & 2 & -1 \\ 2 & -1 & -1 \end{bmatrix}, \quad K_{135} = \begin{bmatrix} 2 & -1 & -1 \\ -1 & 2 & -1 \\ -1 & -1 & 2 \end{bmatrix}$$

1. Tích chập ảnh với từng kernel, hiển thị response map của mỗi hướng.
2. Áp dụng ngưỡng để trích xuất vị trí các đường thẳng.
3. Nhận xét: kernel phát hiện được loại đường thẳng nào? Hạn chế của phương pháp này?

3.2. Phát hiện đường thẳng bằng Hough Transform

1. **Standard Hough Transform** (HoughLines): vẽ các đường thẳng lên ảnh gốc.
2. **Probabilistic Hough Transform** (HoughLinesP): vẽ các đoạn thẳng.
3. Thử ít nhất 2 bộ tham số khác nhau cho mỗi phương pháp.

3.3. So sánh hai phương pháp

1. So sánh kết quả giữa **Convolution Kernel** và **Hough Transform**:
 - Kernel: phát hiện theo hướng cố định, nhạy cảm với nhiễu như thế nào?
 - Hough: phát hiện toàn cục trong không gian tham số – ưu điểm là gì?
2. So sánh Standard vs Probabilistic Hough về số đường phát hiện, độ chính xác và tốc độ.
3. **Nhận xét tổng hợp**: phương pháp nào phù hợp với từng loại ảnh đầu vào?